Exercise 2 Date:

Develop and compare CLI, GUI, and Voice User Interfaces (VUI) for the same task and assess user satisfaction using Python (Tkinter for GUI, Speech Recognition for VUI), Terminal

AIM:

The aim is to develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) for the same task, and assess user satisfaction using Python (with Tkinter for GUI and Speech Recognition for VUI) and Terminal.

PROCEDURE:

i) CLI (Command Line Interface)

CLI implementation where users can add, view, and remove tasks using the terminal.

```
attendance = {}

def add_student(name):
    if name not in attendance:
        attendance[name] = []
        print(f"Student '{name}' added.")
    else:
        print("Student already exists.")

def mark_attendance(name, status):
    if name in attendance:
        attendance[name].append(status)
        print(f"Attendance marked for '{name}'.")
    else:
        print("Student not found.")
```

```
def view_attendance():
    if attendance:
        print("\nAttendance Record:")
        for student, records in attendance.items():
            print(f"{student}: {records}")
    else:
        print("No attendance records found.")

def remove_student(name):
    if name in attendance:
        del attendance[name]
        print(f"Student '{name}' removed.")
    else:
        print("Student not found.")

def main():
    while True:
        print("\nOptions:")
        print("1. Add Student")
        print("2. Mark Attendance")
        print("3. View Attendance")
        print("4. Remove Student")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            name = input("Enter student name: ")
            add_student(name)
```

```
        elif choice == '2':
            name = input("Enter student name: ")
            status = input("Enter status (Present/Absent): ")
            mark_attendance(name, status)

        elif choice == '3':
            view_attendance()

        elif choice == '4':
            name = input("Enter student name to remove: ")
            remove_student(name)

        elif choice == '5':
            print("Exiting...")
            break

        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Sample Output:

```
1. Add Student
2. Mark Attendance
3. View Attendance
4. Remove Student
5. Exit
Enter your choice: 1
Enter student name: Gawin
Student 'Gawin' added.

Options:
1. Add Student
2. Mark Attendance
3. View Attendance
4. Remove Student
5. Exit
Enter your choice: 2
Enter student name: Gawin
Enter status (Present/Absent): p
Attendance marked for 'Gawin'.

Options:
1. Add Student
2. Mark Attendance
3. View Attendance
4. Remove Student
5. Exit
Enter your choice: 3

Attendance Record:
Gawin: ['p']

Options:
1. Add Student
2. Mark Attendance
3. View Attendance
4. Remove Student
5. Exit
Enter your choice: 5
Exiting...
```

ii) GUI (Graphical User Interface)

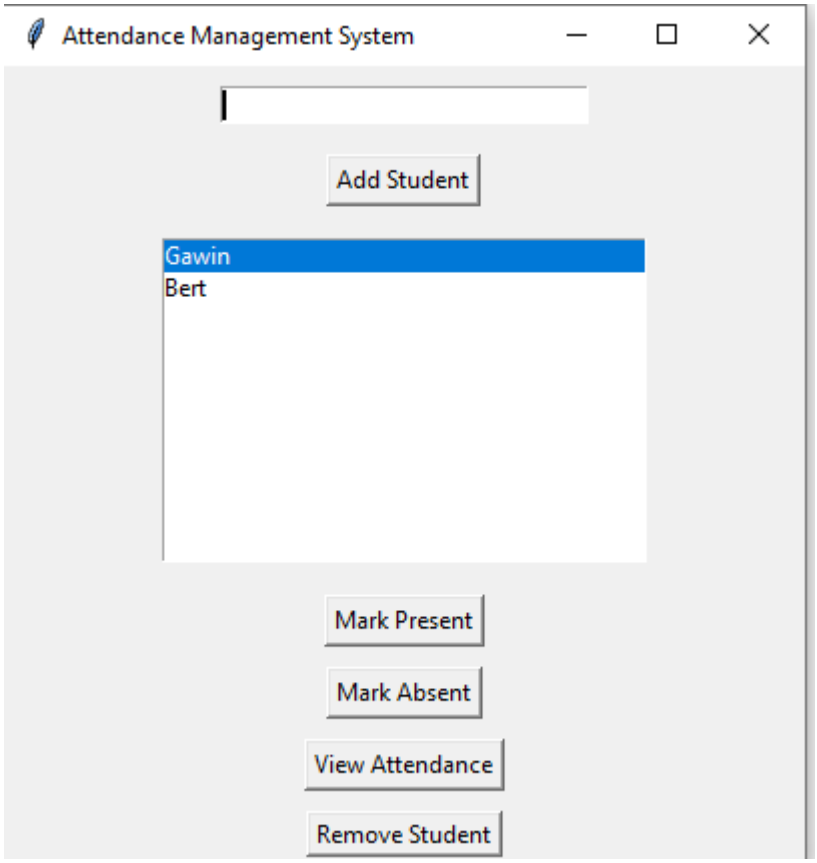
Tkinter to create a simple GUI for our To-Do List application.

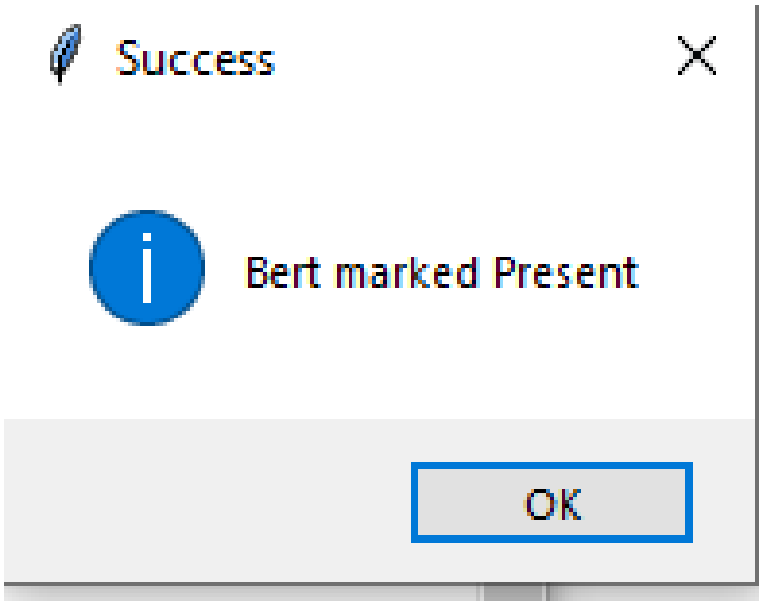
```
import tkinter as tk
from tkinter import messagebox

attendance = {}

def add_student():
    name = student_entry.get()
    if name:
        if name not in attendance:
            attendance[name] = []
            student_list.insert(tk.END, name)
            student_entry.delete(0, tk.END)
        else:
            messagebox.showwarning("Warning", "Student already exists")
    else:
        messagebox.showwarning("Warning", "Student name cannot be empty")

def mark_present():
    selected = student_list.curselection()
    if selected:
        name = student_list.get(selected)
        attendance[name].append("Present")
        messagebox.showinfo("Success", f"{name} marked Present")
    else:
        messagebox.showwarning("Warning", "Select a student")
```

```
def mark_absent():
    selected = student_list.curselection()
    if selected:
        name = student_list.get(selected)
        attendance[name].append("Absent")
        messagebox.showinfo("Success", f"{name} marked Absent")
    else:
        messagebox.showwarning("Warning", "Select a student")

def view_attendance():
    selected = student_list.curselection()
    if selected:
        name = student_list.get(selected)
        records = attendance[name]
        messagebox.showinfo("Attendance Record", f"{name}: {records}")
    else:
        messagebox.showwarning("Warning", "Select a student")

def remove_student():
    selected = student_list.curselection()
    if selected:
        name = student_list.get(selected)
        student_list.delete(selected)
        del attendance[name]
    else:
        messagebox.showwarning("Warning", "Select a student")

# GUI Setup
app = tk.Tk()
```

```
app.title("Attendance Management System")
app.geometry("400x400")

student_entry = tk.Entry(app, width=30)
student_entry.pack(pady=10)

add_button = tk.Button(app, text="Add Student", command=add_student)
add_button.pack(pady=5)

student_list = tk.Listbox(app, width=40, height=10)
student_list.pack(pady=10)

present_button = tk.Button(app, text="Mark Present", command=mark_present)
present_button.pack(pady=5)

absent_button = tk.Button(app, text="Mark Absent", command=mark_absent)
absent_button.pack(pady=5)

view_button = tk.Button(app, text="View Attendance", command=view_attendance)
view_button.pack(pady=5)

remove_button = tk.Button(app, text="Remove Student", command=remove_student)
remove_button.pack(pady=5)

app.mainloop()
```

Sample Output:


Attendance Management System
Add Student
Gawin
Bert
Mark Present
Mark Absent
View Attendance
Remove Student


Success
Bert marked Present
OK

iii) VUI (Voice User Interface)

speech_recognition library for voice input and the pyttsx3 library for text-to-speech output. Make sure you have these libraries installed (pip install SpeechRecognition pyttsx3).

```
import speech_recognition as sr
import pyttsx3

tasks = []

recognizer = sr.Recognizer()
engine = pyttsx3.init()


def speak(text):
    engine.say(text)
    engine.runAndWait()


def add_task(task):
    tasks.append(task)
    speak(f"Task {task} added")


def view_tasks():
    if tasks:
        speak("Your tasks are")
        for i, task in enumerate(tasks, 1):
```

```
        speak(f"Task {i}: {task}")
    else:
        speak("No tasks to show")


def remove_task(task_number):
    if 0 < task_number <= len(tasks):
        removed_task = tasks.pop(task_number - 1)
        speak(f"Task {removed_task} removed")
    else:
        speak("Invalid task number")


def recognize_speech():
    with sr.Microphone() as source:
        print("Listening...")
        audio = recognizer.listen(source)

    try:
        command = recognizer.recognize_google(audio)
        print("You said:", command)
        return command.lower()
    except sr.UnknownValueError:
        speak("Sorry, I did not understand that")
        return None
    except sr.RequestError:
        speak("Network error")
        return None
```

```
def main():
    while True:
        speak("Say add task, view tasks, remove task, or exit")

        command = recognize_speech()

        if not command:
            continue

        if "add task" in command:
            speak("What is the task?")
            task = recognize_speech()
            if task:
                add_task(task)

        elif "view tasks" in command:
            view_tasks()

        elif "remove task" in command:
            speak("Say the task number to remove")
            number = recognize_speech()
            if number and number.isdigit():
                remove_task(int(number))
            else:
                speak("Please say a valid number")

        elif "exit" in command:
            speak("Exiting program")
```

```
        break

    else:
        speak("Invalid option. Please try again")


if __name__ == "__main__":
    main()
```

Sample Output:

```
System: Say add student, mark attendance, view attendance, remove student, or exit
Console: Listening...

User: add student
System: What is the student name?
Console: Listening...
Console: You said: Sarah
System: Student Sarah added successfully

----------------------------------------
```

```
========================================
VOICE ATTENDANCE MANAGEMENT - SAMPLE OUTPUT
========================================

System: Say add student, mark attendance, view attendance, remove student, or exit
Console: Listening...

User: add student
System: What is the student name?
Console: Listening...
Console: You said: John
System: Student John added successfully
```

```
System: Say add student, mark attendance, view attendance, remove student, or exit
Console: Listening...

User: exit
System: Exiting Attendance Management System
```

```
System: Say add student, mark attendance, view attendance, remove student, or exit
Console: Listening...

User: mark attendance
System: Say the student name
Console: Listening...
Console: You said: John
System: Say present or absent
Console: Listening...
Console: You said: present
System: John marked Present
```